CN-Stats-Lec13-Statistical Test for Bivariate Analysis | Case Study: Kabaddi Matches

Statistical Test for Bivariate Analysis | Case Study: Kabaddi Matches

🎯 Session Objectives:

- ✅ Perform and interpret One-Sample T-Tests, ANOVA, and Chi-Square Tests using Python.
- ✅ Understand the purpose and application of the T-test, ANOVA, and Chi-Square tests in data analysis.
- ✅ Formulate and test hypotheses using the T-test to compare means between two groups.
- ✅ Apply ANOVA to compare means across multiple groups and assess the relationship between continuous and categorical variables.
- ✅ Conduct Chi-Square tests to examine associations between categorical variables.
- ✅ Interpret the results of these statistical tests and make data-driven decisions based on the findings.

```
flight_delay['UniqueCarrier'].nunique()
```

22

```
# Carrier Analysis
Question: Does the carrier affect the likelihood of delays?
H0 : Average delay time is the same across all airline carriers.
HA: At least one carrier has significantly different average delay time
```

```
# Annova Testing
flight_delay['dep_delayed_15min'] = flight_delay['dep_delayed_15min'].map({'Y':1 , 'N':0})
```

```
flight_delay.sample(5)
```

	Month	DayofMonth	DayOfWeek	DepTime	UniqueCarrier	Origin	Dest	Distance	dep_delayed_15min
69304	c-3	c-5	c-7	1101	DL	PNS	ATL	272	0
18811	c-7	c-1	c-5	2330	OH	JFK	CHS	636	1
93171	c-8	c-7	c-1	2005	CO	EWR	LAX	2454	1
39457	c-8	c-2	c-3	1210	WN	CMH	BWI	336	0
47376	c-8	c-15	c-2	607	F9	MDW	DEN	895	0

```python
flight_delay['UniqueCarrier'].values
```

```
array(['AA', 'US', 'XE', ..., 'NW', 'MQ', 'WN'], dtype=object)
```

```python
flight_delay['UniqueCarrier'].value_counts()
```

```
UniqueCarrier
WN    15082
AA     9418
DL     8128
MQ     7443
OO     7390
UA     6876
US     6482
NW     6403
XE     5901
OH     4594
CO     4334
EV     3930
FL     3039
AS     2222
YV     2128
B6     1838
```

```python
# Degree of Freedom -> Between  [k-1] # (22-1) # 21
# Degree of Freedom -> Within   [n-k] # (10k-21) # 10k (infinite)
f_critical -> (0.05 ->  1.831 , 0.01 -> 2.321)
```

```python
carrier_group = [group['dep_delayed_15min'].values for name , group in flight_delay.groupby('UniqueCarrier')]
carrier_group
```

```
[array([0, 0, 0, ..., 0, 0, 0], dtype=int64),
 array([1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0,
        0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64),
 array([0, 1, 0, ..., 0, 0, 0], dtype=int64),
 array([0, 0, 0, ..., 0, 0, 0], dtype=int64),
 array([0, 0, 0, ..., 0, 0, 0], dtype=int64),
 array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
```

```python
for name , group in flight_delay.groupby('UniqueCarrier'):
    print(name,group)
```

```
AA       Month DayofMonth DayOfWeek  DepTime UniqueCarrier Origin Dest  Distance  \
0       c-8       c-21       c-7     1934            AA    ATL  DFW       732
8       c-7       c-28       c-5      735            AA    ONT  DFW      1189
13      c-8       c-20       c-6      737            AA    DFW  SJC      1438
20      c-8       c-22       c-2     1521            AA    CLT  DFW       936
26      c-7        c-3       c-7     1350            AA    FLL  LGA      1076
...     ...        ...       ...      ...           ...    ...  ...       ...
99939   c-1       c-31       c-1      656            AA    BOS  MCO      1121
99950   c-1        c-6       c-5     1944            AA    DCA  ORD       612
99952   c-3        c-3       c-4      819            AA    ORD  SJC      1829
99979   c-3       c-13       c-7     2009            AA    DFW  SAT       247
99982   c-8        c-6       c-7     1300            AA    ELP  DFW       551

       dep_delayed_15min
0                      0
8                      0
13                     0
```

```python
from scipy.stats import f_oneway
f_stat , p_value  =  f_oneway(*carrier_group)
f_stat , p_value
```

```
(29.772993358245227, 1.9566717077481475e-118)
```

```
f_emperical (29.77) >> f_critical(1.831 , 2.321))
P-value < 0.05 -> Rejecting the null hypothesis.
```

```python
flight_delay.groupby('UniqueCarrier')['dep_delayed_15min'].sum()
```

```
UniqueCarrier
AA    1770
AQ      18
AS     541
B6     393
CO     788
DH     156
DL    1373
EV    1082
F9     156
FL     736
HA      33
HP     196
MQ    1448
NW    1075
OH     808
OO    1277
```

```
# Day of Week Analysis
Question: Does the day of the week influence flight delays?
H0 : Average delay time is the same for all days of the week.
HA: At least one day has significantly different average delay time.
```

```
day_group = [group['dep_delayed_15min'].values for name , group in flight_delay.groupby('DayOfWeek')]
day_group
```

```
[array([0, 0, 0, ..., 1, 0, 0], dtype=int64),
 array([0, 0, 0, ..., 0, 0, 0], dtype=int64),
 array([0, 1, 0, ..., 0, 0, 0], dtype=int64),
 array([0, 0, 0, ..., 0, 0, 0], dtype=int64),
 array([0, 0, 1, ..., 1, 0, 0], dtype=int64),
 array([0, 1, 0, ..., 0, 0, 1], dtype=int64),
 array([0, 0, 0, ..., 0, 0, 0], dtype=int64)]
```

```
for name , group in flight_delay.groupby('DayOfWeek'):
    print(name,group)
```

```
c-1       Month DayofMonth DayOfWeek  DepTime UniqueCarrier Origin Dest  Distance  \
25      c-4        c-4       c-1      635            WN    SMF  PHX       647
29      c-4       c-18       c-1      619            AA    ORD  LAX      1745
31      c-5        c-9       c-1     1327            AA    DFW  ORF      1213
43      c-4       c-18       c-1     1200            WN    SEA  PHX      1107
53      c-8       c-14       c-1     1519            TZ    PHX  OGG      2845
...     ...        ...       ...      ...           ...    ...  ...       ...
99981   c-8       c-15       c-1     1738            DL    ATL  PDX      2172
99983  c-10       c-24       c-1     1010            DH    PWM  IAD       494
99989   c-4       c-11       c-1     1730            WN    JAN  BWI       888
99991   c-4       c-11       c-1      857            DL    BOS  CVG       752
99992   c-1       c-17       c-1     2009            UA    DEN  BOI       649

       dep_delayed_15min
25                     0
29                     0
31                     0
43                     0
53                     0
...                  ...
99981                  0
99983                  0
99989                  1
```

```python
flight_delay['DayOfWeek'].value_counts()
```

```
DayOfWeek
c-4    14736
c-5    14657
c-2    14604
c-1    14569
c-3    14508
c-7    14097
c-6    12829
Name: count, dtype: int64
```

```python
from scipy.stats import f_oneway
f_stat , p_value  =  f_oneway(*day_group)
f_stat , p_value
```

```
(43.62294460377913, 1.4985733432958586e-53)
```

```
# Degree of Freedom -> Between  [k-1] # (7-1) # 6 (Numerator)
# Degree of Freedom -> Within   [n-k] # 14K -> Infinite (Denominator)
f_critical -> (0.05 ->  2.099 , 0.01 -> 2.802)
f_emperical (43.63) > (2.099 , 2.802)
p_value < 0.05 : Rejecting the Null Hypothesis .... Accepting the Alternate Hypothesis.
```

```python
flight_delay.groupby('DayOfWeek')['dep_delayed_15min'].sum()
```

```
DayOfWeek
c-1    2774
c-2    2451
c-3    2567
c-4    3117
c-5    3285
c-6    2145
c-7    2705
Name: dep_delayed_15min, dtype: int64
```

```python
# Chi-Square Implementation
import pandas as pd
flight_delay = pd.read_csv("flight_delays_train.csv")

contingency_table = pd.crosstab(flight_delay['DayOfWeek'] , flight_delay['dep_delayed_15min'] )
contingency_table
```

dep_delayed_15min	N	Y
DayOfWeek		
c-1	11795	2774
c-2	12153	2451
c-3	11941	2567
c-4	11619	3117
c-5	11372	3285
c-6	10684	2145
c-7	11392	2705

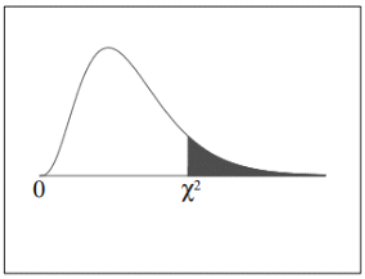
```python
# Chi-Square Implementation
from scipy.stats import chi2_contingency
chi_stats, p_value, df , expected_frequencies = chi2_contingency(contingency_table)
chi_stats, p_value, df , expected_frequencies
```

```
(261.07261732649584,
 1.7615211590667492e-53,
 6,
 array([[11794.47964,  2774.52036],
        [11822.81424,  2781.18576],
        [11745.09648,  2762.90352],
        [11929.67616,  2806.32384],
        [11865.72092,  2791.27908],
        [10385.84524,  2443.15476],
        [11412.36732,  2684.63268]]))
```

```python
chi_stats_emperical [261] > chi_stats_critical(12.592 14.449 16.812 18.548)
# p_value < 0.05 : Rejecting the Null Hypothesis...
```

```
There is a significant association between the day of the week and the likelihood of flight delays.
```

Chi-Square Distribution Table



The shaded area is equal to α for $\chi^2 = \chi^2_\alpha$.

95%-99%

df	$\chi^2_{.995}$	$\chi^2_{.990}$	$\chi^2_{.975}$	$\chi^2_{.950}$	$\chi^2_{.900}$	$\chi^2_{.100}$	$\chi^2_{.050}$	$\chi^2_{.025}$	$\chi^2_{.010}$	$\chi^2_{.005}$
1	0.000	0.000	0.001	0.004	0.016	2.706	3.841	5.024	6.635	7.879
2	0.010	0.020	0.051	0.103	0.211	4.605	5.991	7.378	9.210	10.597
3	0.072	0.115	0.216	0.352	0.584	6.251	7.815	9.348	11.345	12.838
4	0.207	0.297	0.484	0.711	1.064	7.779	9.488	11.143	13.277	14.860
5	0.412	0.554	0.831	1.145	1.610	9.236	11.070	12.833	15.086	16.750
6	0.676	0.872	1.237	1.635	2.204	10.645	12.592	14.449	16.812	18.548
7	0.989	1.239	1.690	2.167	2.833	12.017	14.067	16.013	18.475	20.278
8	1.344	1.646	2.180	2.733	3.490	13.362	15.507	17.535	20.090	21.955
9	1.735	2.088	2.700	3.325	4.168	14.684	16.919	19.023	21.666	23.589
10	2.156	2.558	3.247	3.940	4.865	15.987	18.307	20.483	23.209	25.188

Title: Analyzing Kabaddi Match Events Using Exploratory Data Analysis (EDA)

Objective: *The objective of this case study is to perform an in-depth Exploratory Data Analysis (EDA) on a dataset containing detailed event information from Kabaddi matches. The goal is to gain insights into the gameplay, identify patterns, and uncover relationships between different match events and player performances. Students will learn to clean, manipulate, and visualize data using Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn. Additionally, you will explore the use of group by operations and pivot tables to perform advanced aggregations and percentage calculations.*

Dataset Details: *The dataset that will be used in the class is the events dataset, which contains event-level data from Kabaddi matches with various columns. While we will focus on the events dataset, you are encouraged to explore and apply the same functions and techniques to other datasets available in the Kabaddi database provided in the drive.*

Datasets(Open link in a new tab and download the dataset.)

- ***Unnamed: 0****: Index column*
- ***clock****: Time of the event*
- ***defender_id****: ID of the defender involved in the event*
- ***defenders****: Number of defenders involved in the event*
- ***defending_all_out_points****: Points earned for all-out defense*
- ***defending_bonus_points****: Bonus points earned in defense*
- ***defending_capture_points****: Points earned for capturing the raider*
- ***defending_points****: Total points earned in defense*
- ***defending_team_id****: Team ID of the defending team*
- ***defending_technical_points****: Technical points earned in defense*
- ***do_or_die****: Indicator of a do-or-die raid*
- ***event****: Type of event (e.g., raid, defense)*
- ***event_id****: Unique identifier for the event*
- ***event_no****: Sequential number of the event*
- ***event_text****: Description of the event*
- ***match_id****: Unique identifier for the match*

- ***player_id***: *ID of the player involved in the event*
- ***raid_all_out_points***: *Points earned for all-out raid*
- ***raid_bonus_points***: *Bonus points earned in raid*
- ***raid_points***: *Total points earned in raid*
- ***raid_technical_points***: *Technical points earned in raid*
- ***raid_touch_points***: *Points earned for touching defenders*
- ***raider_bonus***: *Bonus earned by the raider*
- ***raider_id***: *ID of the raider involved in the event*
- ***raiding_team_id***: *Team ID of the raiding team*
- ***reason***: *Reason for the event (if any)*
- ***reason_id***: *Unique identifier for the reason*
- ***review.outcome***: *Outcome of the review (if any)*
- ***review.outcome.id***: *ID of the review outcome*
- ***review.outcome.outcome_value***: *Value of the review outcome*
- ***review.reason***: *Reason for the review (if any)*
- ***review.reason.id***: *ID of the review reason*
- ***review.reason.reason_value***: *Value of the review reason*
- ***review.review_taken***: *Indicator of whether a review was taken*
- ***review.team_id***: *Team ID involved in the review*
- ***score***: *Score of the match at the time of the event*
- ***status_id***: *Status ID of the event*
- ***substituted_by***: *Player who substituted another*
- ***super_raid***: *Indicator of a super raid*
- ***super_tackle***: *Indicator of a super tackle*
- ***team_id***: *ID of the team involved in the event*

Tasks:
The below tasks are general steps taken in an EDA assignment. These steps should be followed when we analyse any dataset.

1. ***Data Loading:***
   1. *Load the dataset using different methods (e.g., CSV, Excel).*
   2. *Display the first few rows to understand the structure of the dataset.*
2. ***Basic Data Exploration:***
   1. *Basic Data Exploration:*
   2. *Generate summary statistics and visualizations to understand the distribution of key variables.*
3. ***Data Cleaning:***
   1. *Handle missing values and inconsistencies in the dataset.*
   2. *Convert data types and ensure data integrity by correcting erroneous values.*
4. ***Data Transformation:***
   1. *Create new features based on existing columns to enhance the dataset.*
   2. *Apply transformations to correct, normalize, or standardize data where necessary.*
5. ***Identifying and Handling Outliers:***
   1. *Identify outliers using visual (box plots) and statistical (IQR) methods.*
   2. *Decide whether to remove or transform outliers and justify your approach.*
6. ***Descriptive Analysis and Visualization:***
   1. *Perform univariate, bivariate, and multivariate analysis using various plots and visualizations.*
   2. *Use correlation analysis to explore relationships between different variables.*
7. ***Group By and Pivot Tables:***
   1. *Use group by operations to calculate aggregated statistics (e.g., mean, sum) based on different grouping variables.*
   2. *Create pivot tables to analyze data from different perspectives, including row-wise and column-wise percentages.*
   3. *Visualize the results using heatmaps to uncover trends and patterns.*
8. ***Advanced Pivot Table Analysis:***
   1. *Explore advanced pivot table functionalities, such as calculating row-wise percentages, column-wise percentages, and percentages of the grand total.*
   2. *Use these techniques to gain deeper insights into player and team performances.*

Tasks:
By the end of this case study, you should be able to:

- *Perform a comprehensive exploratory data analysis (EDA) on a sports dataset.*
- *Clean and transform data to prepare it for analysis.*
- *Identify and handle outliers effectively.*
- *Use group by and pivot tables to perform complex data aggregations and calculations.*
- *Visualize data to communicate insights and trends clearly.*

This case study will help you develop a strong foundation in data analysis and visualization, equipping you with the skills needed to analyze real-world datasets using Python.

https://www.prokabaddi.com/teams

```
# Importing all the dependencies requried for the Case Study ...
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
events_df = pd.read_csv("DS_events.csv", low_memory = False)
events_df.sample(10)
```

	Column1	clock	defender_id	defenders	defending_all_out_points	defending_bonus_points	defending_capture_points	de
8533	88	34:34	NaN	NaN	NaN	NaN	NaN	
15805	50	18:32	2657.0	NaN	0.0	0.0	1.0	
34428	76	06:18	NaN	NaN	NaN	NaN	NaN	
61702	44	18:07	NaN	NaN	0.0	0.0	0.0	
54719	17	12:34	NaN	NaN	NaN	NaN	NaN	
60239	91	01:35	NaN	NaN	NaN	NaN	NaN	
21942	14	06:28	360.0	NaN	NaN	NaN	NaN	
33878	85	08:08	NaN	NaN	NaN	NaN	NaN	
3519	18	08:13	NaN	NaN	NaN	NaN	NaN	
32981	86	04:00	NaN	NaN	NaN	NaN	NaN	

10 rows × 41 columns